国密SMS4密码算法

国密SMS4是一个分组算法。该算法的分组长度为 128 比特,密钥长度为 128 比特。加密算 法与密钥扩展算法都采用 32 轮非线性迭代结构。解密算法与加密算法的结构相同,只是轮 密钥的使用顺序 相反 ,解密轮密钥是加密轮密钥的 逆序 。

需熟读对应的规范文档《无线局域网产品使用的SMS4密码算法(http://www.oscca.gov.cn/sca/c100061/201611/1002423/files/330480f731f64e1ea75138211ea0dc27.pdf)》后，再阅读本文。

术语说明

字与字节 🔗

字节相信大家都比较清楚，一个字节是 8 比特，字即为 32 比特表示的元素，也就是说一个字等于4字节。

代码表示转换关系:

```
// 将const unsigned char *表示的数据转换为字
#define SMS4_GET_UNSIGNED(src, i) (((src)[i * 4] << 24)  | \
        ((src)[i * 4 + 1] << 16) |  \
        ((src)[i * 4 + 2] << 8) |  \
        ((src)[i * 4 + 3]))

//将字转换为const unsigned char *表示的数据
#define SMS4_GET_BE(src) ((((src) & 0x0FF) << 24) | \
        ((((src) >> 8) & 0x0FF) << 16) | \
        ((((src) >> 16) & 0x0FF) << 8) | \
        ((((src) >> 24) & 0x0FF)))
```

S盒

S 盒为固定的 8 比特输入 8 比特输出的置换,记为 Sbox(.)。

代码表示:

```
unsigned char SMS4_SBOX[256] ={
    0xd6, 0x90, 0xe9, 0xfe, 0xcc, 0xe1, 0x3d, 0xb7, 0x16, 0xb6, 0x14, 0xc2, 0x28,
0xfb, 0x2c, 0x05,
    0x2b, 0x67, 0x9a, 0x76, 0x2a, 0xbe, 0x04, 0xc3, 0xaa, 0x44, 0x13, 0x26, 0x49,
0x86, 0x06, 0x99,
    0x9c, 0x42, 0x50, 0xf4, 0x91, 0xef, 0x98, 0x7a, 0x33, 0x54, 0x0b, 0x43, 0xed,
0xcf, 0xac, 0x62,
    0xe4, 0xb3, 0x1c, 0xa9, 0xc9, 0x08, 0xe8, 0x95, 0x80, 0xdf, 0x94, 0xfa, 0x75,
0x8f, 0x3f, 0xa6,
    0x47, 0x07, 0xa7, 0xfc, 0xf3, 0x73, 0x17, 0xba, 0x83, 0x59, 0x3c, 0x19, 0xe6,
0x85, 0x4f, 0xa8,
    0x68, 0x6b, 0x81, 0xb2, 0x71, 0x64, 0xda, 0x8b, 0xf8, 0xeb, 0x0f, 0x4b, 0x70,
0x56, 0x9d, 0x35,
    0x1e, 0x24, 0x0e, 0x5e, 0x63, 0x58, 0xd1, 0xa2, 0x25, 0x22, 0x7c, 0x3b, 0x01,
0x21, 0x78, 0x87,
    0xd4, 0x00, 0x46, 0x57, 0x9f, 0xd3, 0x27, 0x52, 0x4c, 0x36, 0x02, 0xe7, 0xa0,
0xc4, 0xc8, 0x9e,
    0xea, 0xbf, 0x8a, 0xd2, 0x40, 0xc7, 0x38, 0xb5, 0xa3, 0xf7, 0xf2, 0xce, 0xf9,
0x61, 0x15, 0xa1,
    0xe0, 0xae, 0x5d, 0xa4, 0x9b, 0x34, 0x1a, 0x55, 0xad, 0x93, 0x32, 0x30, 0xf5,
0x8c, 0xb1, 0xe3,
    0x1d, 0xf6, 0xe2, 0x2e, 0x82, 0x66, 0xca, 0x60, 0xc0, 0x29, 0x23, 0xab, 0x0d,
0x53, 0x4e, 0x6f,
    0xd5, 0xdb, 0x37, 0x45, 0xde, 0xfd, 0x8e, 0x2f, 0x03, 0xff, 0x6a, 0x72, 0x6d,
0x6c, 0x5b, 0x51,
    0x8d, 0x1b, 0xaf, 0x92, 0xbb, 0xdd, 0xbc, 0x7f, 0x11, 0xd9, 0x5c, 0x41, 0x1f,
0x10, 0x5a, 0xd8,
    0x0a, 0xc1, 0x31, 0x88, 0xa5, 0xcd, 0x7b, 0xbd, 0x2d, 0x74, 0xd0, 0x12, 0xb8,
0xe5, 0xb4, 0xb0,
    0x89, 0x69, 0x97, 0x4a, 0x0c, 0x96, 0x77, 0x7e, 0x65, 0xb9, 0xf1, 0x09, 0xc5,
0x6e, 0xc6, 0x84,
    0x18, 0xf0, 0x7d, 0xec, 0x3a, 0xdc, 0x4d, 0x20, 0x79, 0xee, 0x5f, 0x3e, 0xd7,
0xcb, 0x39, 0x48
};
```

基本运算

算法中采用了以下基本运算:

```
⊕  32 比特异或
<<< i 32 比特循环左移 i 位
```

代码表示循环左移

```
#define CROL(x, b) ((((x) & 0xFFFFFFFF) << (b)) | ((x) >> (0x20 - (b))))
```

加/解密运算

> 定义反序变换 R 为:
> $R(A_0, A_1, A_2, A_3) = (A_3, A_2, A_1, A_0)$。
>
> 设明文输入为 (X_0, X_1, X_2, X_3), 密文输出为 (Y_0, Y_1, Y_2, Y_3) 轮密钥为 rk_i, $i = 0, 1, 2, \ldots, 31$。
>
> 则加密变换为: $X_{i+4} = F(X_i, X_{i+1}, X_{i+2}, X_{i+3}, rk_i) = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)$, $i = 0, 1, \ldots, 31$.
> $(Y_0, Y_1, Y_2, Y_3) = R(X_{32}, X_{33}, X_{34}, X_{35}) = (X_{35}, X_{34}, X_{33}, X_{32})$。
>
> 解密变换与加密变换结构相同, 不同的仅是轮密钥的使用顺序。
> 加密时轮密钥的使用顺序为:$(rk_0, rk_1, \ldots, rk_{31})$
> 解密时轮密钥的使用顺序为:$(rk_{31}, rk_{30}, \ldots, rk_0$

规范中的这段要折细来理解的话其实也很好理解，首先假定了输入是 (X_0, X_1, X_2, X_3)，这里的X_i, $i=0,1,2,3$是字表示，也就是说一次加密变换只能操作4个字，也就是16个字节。 通常的C/C++中我们都会用const unsigned char *来指向输入的数据，那么如何转换为 (X_0, X_1, X_2, X_3)呢，我们来逐步实现：

```
/**
 * 我们假定我们的加/解密变换的定义如下，这里不考虑in及out是否为空指针或内存会不会
溢出，由业务层处理。
 * @param key 密钥
 * @param mode 0表示加密，1表示解密
 * @param in 表示输入数据
 * @param out 表示输出数据
 */
void sms4_crypt(const unsigned char *key, int mode, const unsigned char *in, unsi
gned char *out)
{
    unsigned int X[36]; // 定义输入及变换过程暂存字缓冲区
    int i;
    // 将in转换为字，存入X缓冲区
    for(i = 0; i < 4; i++)
    {
        X[i] = SMS4_GET_UNSIGNED(in, i);
    }
}
```

加/解密运算后的结果用(Y_0, Y_1, Y_2, Y_3)，而$(Y_0, Y_1, Y_2, Y_3) = R(X_{32}, X_{33}, X_{34}, X_{35}) = (X_{35}, X_{34}, X_{33}, X_{32})$。那么完善我们的加密算法

```
void sms4_crypt(const unsigned char *key, int mode, const unsigned char *in, unsi
gned char *out)
{
    unsigned int X[36]; // 定义输入及变换过程暂存字缓冲区
    int i;
    unsigned int * pout = (unsigned int *)out;

    // 将in转换为字，存入X缓冲区
    for(i = 0; i < 4; i++)
    {
        X[i] = SMS4_GET_UNSIGNED(in, i);
    }

    //将运算结果输出
    pout[0] = SMS4_GET_BE(X[35]);
    pout[1] = SMS4_GET_BE(X[34]);
    pout[2] = SMS4_GET_BE(X[33]);
    pout[3] = SMS4_GET_BE(X[32]);
}
```

接下来加密的过程就是如何一步步计算 X_4，X_5，X_6,...,X_{35}

规范中的算法是
$X_{i+4} = F(X_i, X_{i+1}, X_{i+2}, X_{i+3}, rk_i) = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)$, $i = 0, 1, \ldots, 31$.

也即$X_4=X_0 \wedge T(X_1 \wedge X_2 \wedge X_3 \wedge rk_i)$，$X_{0-3}$都已经有了，剩下T变换和$rk_i$。

rk是轮密钥，规范中提到，如果是加密则$i=0, 1, \ldots, 31$，反之解密为$i=31, 30, \ldots, 0$。 也就是：

> 加密： $X_4=X_0 \wedge T(X_1 \wedge X_2 \wedge X_3 \wedge rk_0)$
> 解密： $X_4=X_0 \wedge T(X_1 \wedge X_2 \wedge X_3 \wedge rk_{31})$

以此类推计算 X_5,X_6 ,..., X_{35} ，整个加/解密运算要经过32轮变换。

设: $tmp_1=X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i$, tmp_2 为非线性变换后的结果，那么加/解密运算的代码实现如下：

```
void sms4_crypt(const unsigned char *key, int mode, const unsigned char *in, unsi
gned char *out)
{
    unsigned int X[36]; // 定义输入及变换过程暂存字缓冲区
    unsigned int tmp1;
    int i;
    unsigned int * pout = (unsigned int *)out;

    // 将in转换为字，存入X缓冲区
    for(i = 0; i < 4; i++)
    {
        X[i] = SMS4_GET_UNSIGNED(in, i);
    }

    sms4_key_schedule(key, rk);

    for(i = 0; i < 32; i++)
    {
        tmp1 = X[i + 1] ^ X[i + 2] ^ X[i + 3] ^ rk[( (mode == 0) ? i : 31 - i )];

        // non linear operation
        SMS4_NON_LINEAR_OP(tmp1, tmp2);

        X[i + 4] = X[i] ^ (tmp2 ^ CROL(tmp2, 2) ^ CROL(tmp2, 10) ^ CROL(tmp2, 18)
^ CROL(tmp2, 24));
    }

    //将运算结果输出
    pout[0] = SMS4_GET_BE(X[35]);
    pout[1] = SMS4_GET_BE(X[34]);
    pout[2] = SMS4_GET_BE(X[33]);
    pout[3] = SMS4_GET_BE(X[32]);
}
```

Copyright © 2012-2019 yiim.net Inc. All Rights Reserved. 粤ICP备16069452号-2 (http://www.miitbeian.gov.cn/)

Powered By Saint.